### 7.4.6 Letting the AS decrypt the tokens

An alternative to using public key signing and encryption would be for the AS to encrypt access tokens with a symmetric authenticated encryption algorithm, such as the ones you learned about in chapter 6. Rather than sharing this symmetric key with every API, they would instead call the token introspection endpoint to validate the token rather than verifying it locally. Because the AS does not need to perform a database lookup to validate the token, it may be easier to horizontally scale the AS in this case by adding more servers to handle increased traffic.

This pattern allows the format of access tokens to change over time because only the AS validates tokens. In software engineering terms, the choice of token format is encapsulated by the AS and hidden from resource servers, while with public key signed JWTs each API knows how to validate tokens making it much harder to change the representation later. More sophisticated patterns for managing access tokens for microservice environments are covered in part 3.

## 7.5 Single sign-on

One of the advantages of OAuth2 is the ability to centralize authentication of users at the AS, providing a single sign-on (SSO) experience (figure 7.10). When the user's client needs to access an API, it redirects the user to the AS authorization endpoint to get an access token. At this point the AS